Appunti e Risoluzione di Problemi di Intelligenza Artificiale

Sintesi degli Esercizi

Indice

1 Problema 1: Il Cavaliere, la Principessa e il Drago **2**
- 1.1 Definizione dell'Ambiente Operativo (PEAS) . . . 2
- 1.2 Formulazione dello Spazio degli Stati . . . 2
- 1.3 Strategia di Risoluzione e Algoritmo . . . 2
- 1.4 Pseudocodice . . . 3

2 Problema 2: Robot Assemblatore **4**
- 2.1 Definizione dell'Ambiente Operativo (PEAS) . . . 4
- 2.2 Formulazione dello Spazio degli Stati . . . 4
- 2.3 Strategia di Risoluzione e Algoritmo . . . 4
- 2.4 Pseudocodice dell'Agente . . . 4

3 Problema 3: Taxi Autonomo **6**
- 3.1 Definizione dell'Ambiente Operativo (PEAS) . . . 6
- 3.2 Formulazione dello Spazio degli Stati . . . 6
- 3.3 Strategia di Risoluzione e Algoritmo . . . 6
- 3.4 Pseudocodice dell'Agente . . . 6

1 Problema 1: Il Cavaliere, la Principessa e il Drago

Una principessa è stata rinchiusa in un labirinto (o grotta) insieme a un drago dormiente che le fa da guardia. Il cavaliere deve entrare, aggirare il drago senza svegliarlo, prendere la principessa in braccio e portarla all'uscita.

1.1 Definizione dell'Ambiente Operativo (PEAS)

- **Performance (P):** Minimizzare il costo del cammino per raggiungere la principessa e successivamente l'uscita, evitando il drago.
- **Environment (E):** Labirinto/Grotta, principessa, drago, cavaliere. L'ambiente è composto da una griglia (es. 36 celle), con un punto di entrata/uscita.
- **Actuators (A):** Gambe e braccia del cavaliere (movimento e presa).
- **Sensors (S):** Occhi/Vista (percezione visiva dell'ambiente, le luci sono accese).

Caratteristiche dell'ambiente: Completamente osservabile, a singolo agente (gli altri sono statici), deterministico, discreto e noto.

1.2 Formulazione dello Spazio degli Stati

- **Stati:** La posizione del cavaliere nella griglia. Considerando una griglia di 36 celle e l'assenza/presenza della principessa in braccio, si hanno $36 \times 2 = 72$ stati totali possibili.
- **Stato Iniziale:** Il cavaliere si trova all'entrata E (es. coordinata 1,1). Nel secondo step, lo stato iniziale diventerà la cella in cui si trova la principessa.
- **Stati Obiettivo:** Il primo obiettivo è raggiungere la cella della principessa. Il secondo obiettivo è tornare all'uscita con la principessa.
- **Azioni:** Movimenti cardinali {N, S, E, W} (o {Su, Giù, Destra, Sinistra}), più l'azione `prendi()` e l'azione `esci()`.
- **Modello di Transizione:** L'esecuzione di un'azione di movimento sposta il cavaliere nella cella adiacente desiderata, se non ci sono ostacoli insormontabili. Esempio: $RISULTATO(s, N) = s_{0,1}$.
- **Costo delle Azioni:** Spostarsi in una cella vuota costa $+1$. Spostarsi in una cella con il drago ha costo $+\infty$ (questo rilassamento garantisce che l'algoritmo eviti il drago). Le azioni `prendi()` ed `esci()` hanno costo 0.

1.3 Strategia di Risoluzione e Algoritmo

Essendo l'ambiente noto e deterministico, il problema può essere risolto offline. Gli algoritmi non informati (BFS, DFS) potrebbero esplorare nodi pericolosi o essere inefficienti. Si adotta l'algoritmo **A*** richiamato due volte:

1. Da Entrata a Principessa.
2. Da Principessa a Entrata.

Euristica $h(n)$**:** Distanza di Manhattan. Per due punti $A(x_1, y_1)$ e $B(x_2, y_2)$, $M = |x_2 - x_1| + |y_2 - y_1|$.

1.4 Pseudocodice

```
function A*(problem) returns solution or failure:
    nodo = problem.StatoIniziale
    frontiera = coda di priorità basata su f(n) = g(n) + h(n)
    esplorato = set()
    frontiera.push(nodo)

    while not frontiera.isEmpty():
        nodo = frontiera.pop()
        if nodo.stato == goal:
            return solution
        esplorato.add(nodo)

        for each azione in nodo.azioni:
            figlio = nodo_figlio(nodo, azione, problem)
            if figlio not in frontiera and figlio not in esplorato:
                frontiera.push(figlio)
            else if figlio in frontiera ma con f(n) maggiore:
                sostituisci il nodo in frontiera con figlio (che ha f(n) minore)

    return failure
```

2 Problema 2: Robot Assemblatore

Un robot deve assemblare mobili recuperando i materiali da un magazzino esterno. Può trasportare un solo materiale per volta, effettuando più viaggi. Finiti i materiali, assembla l'oggetto e passa a un nuovo ordine.

2.1 Definizione dell'Ambiente Operativo (PEAS)

- **Performance (P):** Minimizzare il costo delle azioni per recuperare i materiali e svolgere tutti gli ordini.
- **Environment (E):** Magazzino, area operativa, ordini. Ambiente noto, deterministico e osservabile.
- **Actuators (A):** Braccia e gambe (cingoli/ruote) del robot.
- **Sensors (S):** Sensori di prossimità, rilevamento oggetti.

2.2 Formulazione dello Spazio degli Stati

- **Stati:** Dato un ambiente con n posizioni distinte e un ordine che richiede m materiali, il robot può trasportare $0, 1, ..., m$ materiali. Si considera anche lo stato in cui "costruisce" e lo stato in cui è libero. Totale stati stimati: $(n \times (m + 1)) + 2$.
- **Stato Iniziale:** La posizione in cui il robot ha terminato la mansione precedente.
- **Stati Obiettivo:** Costruzione dell'oggetto terminata.
- **Azioni:** Movimento {Destra, Sinistra, Su, Giù}, `prendi()`, `rilascia()`, `costruisci()`.
- **Modello di Transizione:** Movimenti logici nella griglia. `prendi()` preleva un pezzo dal magazzino; `rilascia()` posa il pezzo nell'area di costruzione.
- **Costo delle Azioni:** Costo uniforme ($+1$) per i movimenti, costo 0 per prendere/rilasciare/costruire.

2.3 Strategia di Risoluzione e Algoritmo

Si utilizza una sequenza di ricerche con l'algoritmo **A***. Il piano viene calcolato per ogni materiale: viaggio verso il magazzino e viaggio verso l'area di assemblaggio.
Euristica $h(n)$**:** Distanza Euclidea (in linea d'aria), ideale per ambienti non strutturati. $E = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$. L'euristica è consistente e ammissibile, garantendo l'ottimalità di A*.

2.4 Pseudocodice dell'Agente

```
function robot(stato, goal) returns action:
    persistent plan, state, goal, problem
    persistent ordini (coda FIFO)
    persistent estrazione_ordini (insieme vuoto)

    if estrazione_ordini.isEmpty():
        estrazione_ordini.pop(ordini)
```

```
if plan is empty:
    goal = formulate_goal()
    problem = formulate_problem()
    plan = search(estrazione_ordini) // Ricerca ripetuta per ogni materiale

    if plan is failure:
        return null action

if stato == goal and ha_materiale:
    return rilascia()
if stato == goal and tutti_materiali_raccolti:
    return costruisci()

action = first(plan)
plan = rest(plan)
return action
```

3 Problema 3: Taxi Autonomo

Un veicolo autonomo (taxi) deve pianificare ed eseguire percorsi in una città per prelevare e trasportare i passeggeri dal punto di partenza a quello di destinazione minimizzando la distanza percorsa.

3.1 Definizione dell'Ambiente Operativo (PEAS)

- **Performance (P):** Minimizzare il cammino del taxi all'interno della città per arrivare da un punto all'altro.
- **Environment (E):** Città in cui opera il taxi, mappa stradale.
- **Actuators (A):** Ruote dell'auto, motore, portiere.
- **Sensors (S):** Radar, GPS, sensori dell'auto.

Caratteristiche dell'ambiente: Completamente osservabile (il taxi conosce la mappa), singolo agente (nell'astrazione di base del pathfinding), deterministico, discreto (incroci come nodi) e sequenziale.

3.2 Formulazione dello Spazio degli Stati

- **Stati:** La posizione dell'agente all'interno della mappa della città, combinata con lo stato del passeggero (da prelevare o a bordo).
- **Stato Iniziale:** Il punto in cui si trova il taxi al momento della richiesta.
- **Stato Obiettivo:** Il punto di destinazione richiesto dal passeggero (dopo averlo prelevato).
- **Azioni:** `ACCEPT` (accetta chiamata), `DECLINE` (rifiuta), `OPEN` (apri porte per far scendere/salire il passeggero) e azioni di movimento lungo gli archi.
- **Modello di Transizione:** Un grafo in cui i nodi sono le località (incroci) e gli archi i collegamenti stradali.
- **Costo delle Azioni:** I pesi degli archi rappresentano la distanza in Km o il tempo di percorrenza.

3.3 Strategia di Risoluzione e Algoritmo

L'agente utilizza la ricerca informata **A*** per trovare un percorso ottimale sulla mappa.
Euristica $h(n)$**:** Distanza in linea d'aria (Euclidea) dal nodo n al nodo obiettivo. La funzione di valutazione sarà $f(n) = g(n) + h(n)$, dove $g(n)$ è il costo del percorso effettivo stradale dalla radice.

3.4 Pseudocodice dell'Agente

```
function taxi_agent(percept, goal) returns action:
    persistent plan (vuoto), state, goal, problem, busy

    // Gestione chiamate
    if not busy:
```

```
    ACCEPT()
else:
    DECLINE()

if plan is empty:
    goal = formulate_goal()
    problem = formulate_problem()
    plan = A*(problem)

    if plan is failure:
        return null action

    busy = true

if current_location == goal:
    busy = false
    return OPEN()
else:
    action = first(plan)
    plan = rest(plan)
    return action
```